

Systemy czasu rzeczywistego, Projekt 5

Piotr Gugnowski, wydział EiTI, nr indeksu 320690

Czerwiec 2026

Zadanie 5.1: Standard IEEE 754: zakresy obu systemów kodowania

Treść zadania

Wyznaczyć zakresy liczb w obu systemach kodowania: kodzie "wewnętrznym" oraz w standardzie IEEE 754.

Analiza struktury bitowej

Oba formaty są 32-bitowe. Struktura IEEE 754 single precision: 1 bit znaku, 8 bitów wykładnika z nadmiarem 127, 23 bity mantysy z niejawnym bitem wiodącym.

Struktura kodu wewnętrznego **nie jest wprost podana w treści zadania**. Zadanie 5.1 wymaga jej jednoznacznego wyznaczenia przez analizę wzorców bitowych z tabeli konwersji. Wymaganiem jest, aby format obsługiwał **liczby ujemne oraz wartości mniejsze od 1** (ułamki), dokładnie tak jak IEEE 754.

Format IEEE 754 single precision

Bit 31	Bity 30-23	Bity 22-0
znak S (1 bit)	wykładnik ze stałą E_b (8 bitów)	mantysa M (23 bity)

Wartość liczby znormalizowanej:

$$(-1)^S \times (1 + M \cdot 2^{-23}) \times 2^{E_b - 127} \quad E_b \in [1, 254]$$

Stała (bias) wykładnika wynosi 127.

Kod "wewnętrzny"

Na podstawie tabeli konwersji (np. $1 \rightarrow 0x80000000$, $9 \rightarrow 0x83100000$, $65535 \rightarrow 0x8F7FFF00$) wyznaczono następującą strukturę bitową:

Bity 31-24	Bit 23	Bity 22-0
wykładnik ze stałą E_b (8 bitów, bias 128)	znak S (1 bit)	mantysa M (23 bity)

Wartość liczby znormalizowanej:

$$(-1)^S \times (1 + M \cdot 2^{-23}) \times 2^{E_b - 128} \quad E_b \in [1, 254], \quad M \in [0, 2^{23} - 1]$$

Wyznaczenie nadmiaru z tabeli. Powyższą strukturę (w tym wartość nadmiaru) odczytujemy wprost z danych, bez żadnych założeń. Po pierwsze, mantysa (bity 22-0) jest w obu kodach identyczna, co widać np. dla liczby 9 (mantysa = 0x100000 zarówno w 0x83100000, jak i 0x41100000) oraz 65535 (0x7FFF00 w obu). Pozostaje wyznaczyć rolę najstarszych bitów. Dla każdej liczby z tabeli znamy jej rzeczywisty wykładnik E (z postaci znormalizowanej $1, M \times 2^E$). Porównajmy go z 8-bitowym polem w bitach [31:24] kodu wewnętrznego oraz, dla kontroli, z polem wykładnika IEEE 754 (bity [30:23], o znanym nadmiarze 127):

Liczba	E	wewn. [31:24]	wewn. $-E$	IEEE [30:23]	IEEE $-E$
1	0	0x80 = 128	128	0x7F = 127	127
2	1	0x81 = 129	128	0x80 = 128	127
9	3	0x83 = 131	128	0x82 = 130	127
65535	15	0x8F = 143	128	0x8E = 142	127
65536	16	0x90 = 144	128	0x8F = 143	127

Tabela 1: Wyprowadzenie nadmiaru. Pole [31:24] kodu wewnętrznego pomniejszone o rzeczywisty wykładnik daje *stałą* 128 dla każdej liczby.

Kolumna wewn. $-E$ jest stała i równa 128 w każdym wierszu. Dowodzi to, że bity [31:24] tworzą 8-bitowe pole wykładnika ze stałą (nadmiarem) **128**. Analogiczna kolumna dla IEEE 754 jest stała i równa 127 (znany nadmiar standardu). Różnica nadmiarów wynosi dokładnie 1 i to właśnie ona przekłada się dalej na relację wartości (połowa) i prostą konwersję. Bit 23 we wszystkich przykładach jest zerem (liczby dodatnie); jako jedyny wolny bit między wykładnikiem a mantysą pełni rolę **bitu znaku**, analogicznie do bitu 31 w IEEE 754, co umożliwia liczby ujemne.

Format jest zatem **w pełni analogiczny do IEEE 754**: definiuje te same klasy liczb i rezerwuje te same wartości pola wykładnika. Jedyne różnice to wyznaczone wyżej dwa parametry: nadmiar (128 zamiast 127) oraz położenie bitu znaku (bit 23 zamiast bitu 31).

E_b	M	Znaczenie (dokładnie jak w IEEE 754)
0	0	zero ze znakiem: $+0 = 0x00000000$, $-0 = 0x00800000$
0	$\neq 0$	liczba zdenormalizowana: $(-1)^S \times (M \cdot 2^{-23}) \times 2^{-127}$
1...254	dowolna	liczba znormalizowana: $(-1)^S \times (1 + M \cdot 2^{-23}) \times 2^{E_b-128}$
255	0	$\pm\infty$
255	$\neq 0$	NaN

Relacja wartości do IEEE 754 (klucz do konwersji)

Ponieważ jedyną różnicą *liczbową* jest nadmiar (większy o 1), wartość słowa kodu wewnętrznego jest dokładnie **połową** wartości słowa IEEE 754 zbudowanego z tych samych pól (znak, wykładnik, mantysa):

$$\text{wartość}_{\text{wew}}(X) = \frac{1}{2} \text{wartość}_{\text{IEEE}}(\text{te same pola}).$$

Tożsamość ta zachodzi dla *wszystkich* klas liczb (sprawdzenie: dla znormalizowanej $2^{E_b-128} = 2^{E_b-127}/2$; dla zdenormalizowanej $2^{-127} = 2^{-126}/2$; a $\pm\infty/2 = \pm\infty$, $0/2 = 0$). Dzięki niej konwersja w obie strony sprowadza się do przeniesienia bitu znaku i pomnożenia (lub podzielenia) wartości przez 2, co automatycznie zachowuje semantykę IEEE 754 dla znaku zera, liczb zdenormalizowanych, Inf/NaN, nadmiaru i niedomiaru.

Przypadek specjalny: zero

Zero jest przypadkiem szczególnym, bo nie da się go zapisać w postaci znormalizowanej $1, M \times 2^E$ (brak niejawniej jedynki wiodącej). Tak jak w IEEE 754, kod wewnętrzny rezerwuje na nie $E_b = 0$ (przy $M = 0$) i **rozdziela znak zera**: $+0 = 0x00000000$ oraz $-0 = 0x00800000$ (ustawiony bit znaku 23). Konwersja zachowuje znak zera w obie strony: $+0 \leftrightarrow 0x00000000$, $-0_{\text{IEEE}} (0x80000000) \leftrightarrow -0_{\text{wew}} (0x00800000)$. W przykładach z tabeli konwersji występuje wyłącznie $+0$.

Wszystkie liczby z tabeli to dodatnie liczby całkowite, więc nie pokazują jeszcze obsługi znaku ani ułamków, lecz wynika ona wprost ze struktury:

- **liczby ujemne**: bit znaku $S = 1$ (np. $-9 = 0x83900000$),
- **ułamki** < 1 : wykładnik ze stałą $E_b < 128$, czyli $E < 0$ (np. $0,5 = 0x7F000000$).

Weryfikacja na przykładach z tabeli

Liczba	Kod wewn.	S	E_b	E	M	IEEE 754
0	0x00000000	n/d	n/d	n/d	n/d	0x00000000
1	0x80000000	0	128	0	0	0x3F800000
2	0x81000000	0	129	1	0	0x40000000
9	0x83100000	0	131	3	0x100000	0x41100000
65535	0x8F7FFF00	0	143	15	0x7FFF00	0x477FFF00
65536	0x90000000	0	144	16	0	0x47800000
<i>przykłady spoza tabeli, demonstracja ułamków i liczb ujemnych:</i>						
0,5	0x7F000000	0	127	-1	0	0x3F000000
-1	0x80800000	1	128	0	0	0xBF800000
-9	0x83900000	1	131	3	0x100000	0xC1100000

Tabela 2: Weryfikacja struktury bitowej. Dla każdej liczby $E_{b,\text{IEEE}} = E_b - 1$, a mantysa M jest identyczna w obu formatach.

Przykładowo dla $9 = 1,001_2 \times 2^3$:

$$S = 0, \quad E = 3 \Rightarrow E_b = 3 + 128 = 131 = 0x83, \quad M = 0,001_2 \Rightarrow M \cdot 2^{23} = 2^{20} = 0x100000$$

$$\text{wew} = (0x83 \ll 24) \mid (0 \ll 23) \mid 0x100000 = 0x83100000 \checkmark$$

$$E_{b,\text{IEEE}} = 131 - 1 = 130 = 0x82 \Rightarrow \text{IEEE} = (130 \ll 23) \mid 0x100000 = 0x41100000 \checkmark$$

Dla liczby ujemnej -9 wystarczy ustawić bit znaku (bit 23): $\text{wew} = 0x83100000 \mid 0x00800000 = 0x83900000$.

Zakresy

Kod "wewnętrzny"

Parametr	Wartość
Zakres	$[-(2 - 2^{-23}) \cdot 2^{126}, (2 - 2^{-23}) \cdot 2^{126}]$
Maksymalna wartość	$(2 - 2^{-23}) \times 2^{126} \approx 1,701 \times 10^{38}$
Min. wartość dodatnia znormalizowana	$2^{-127} \approx 5,877 \times 10^{-39}$
Min. wartość dodatnia zdenormalizowana	$2^{-150} \approx 7,006 \times 10^{-46}$
Zakres wykładnika E (znorm.)	$[-127, 126]$ (8 bitów, bias 128)
Zakres mantysy M	$[0, 2^{23} - 1]$ (23 bity)
Liczby ujemne / ułamki / Inf / NaN	tak (jak IEEE 754)

Tabela 3: Zakres kodu "wewnętrznego".

Standard IEEE 754 single precision

Parametr	Wartość
Zakres	$[-(2 - 2^{-23}) \cdot 2^{127}, (2 - 2^{-23}) \cdot 2^{127}]$
Minimalna wartość dodatnia (znormalizowana)	$2^{-126} \approx 1,175 \times 10^{-38}$
Maksymalna wartość	$(2 - 2^{-23}) \times 2^{127} \approx 3,403 \times 10^{38}$
Zakres wykładnika	$E \in [-126, +127]$ (bias = 127)
Zakres mantysy M	$[0, 2^{23} - 1]$ (23 bity)
Liczby ujemne	tak (bit znaku, bit 31)

Tabela 4: Zakres standardu IEEE 754 single precision (32-bit).

Porównanie zakresów

Cecha	Kod wewnętrzny	IEEE 754
Liczby ujemne	tak (bit 23)	tak (bit 31)
Wartości < 1	tak	tak
Zero ze znakiem (± 0)	tak	tak
Liczby zdenormalizowane	tak	tak
Inf / NaN	tak	tak
Bity wykładnika	8 (bias 128)	8 (bias 127)
Bity mantysy	23	23
Zakres wykładnika E (znorm.)	$[-127, 126]$	$[-126, 127]$
Maksymalna wartość	$\approx 1,70 \times 10^{38}$	$\approx 3,40 \times 10^{38}$

Tabela 5: Porównanie cech obu systemów kodowania.

Oba formaty są **identyczne semantycznie**: te same klasy liczb (zero ze znakiem, zdenormalizowane, znormalizowane, Inf/NaN), te same zarezerwowane wartości wykładnika, to samo 23-bitowe pole mantysy. Jedyna różnica liczbową to nadmiar: bias 128 przesunął cały zakres wykładnika o jedną binadę w dół względem IEEE 754 ($E \in [-127, 126]$ zamiast $[-126, 127]$). W praktyce kod wewnętrzny sięga nieco niżej przy małych wartościach, a IEEE 754 nieco wyżej przy dużych (maksimum większe o czynnik 2). Jest to bezpośrednia, nieunikniona konsekwencja nadmiaru 128 wynikającego z tabeli konwersji w treści zadania.

Zadanie 5.2: Konwersja kodu wewnętrznego do IEEE 754

Treść zadania

Napisać aplikację konwersji kodu "wewnętrznego" liczb zmiennoprzecinkowych do kodu liczb zmiennoprzecinkowych zgodnych z IEEE 754. Aplikacja ma umożliwiać wprowadzenie z klawiatury liczb w kodzie binarnym naturalnym.

Implementacja

Aplikacja napisana w C++17, kompilowalna na systemach Windows, macOS i Linux (clang++ lub g++). Do budowania wymagany jest `just` (`just build`). Kod źródłowy składa się z dwóch plików:

- `src/shared.cpp` / `src/shared.h`: funkcje wspólne (konwersje, wyświetlanie, parsowanie wejścia),
- `src/task2_internal_to_ieee.cpp`: punkt wejścia (`main`).

Format wejścia

Zgodnie z założeniem zadania użytkownik wprowadza kod wewnętrzny jako **32-bitowy ciąg w kodzie binarnym naturalnym** (ciąg znaków 0/1). Spacje są dozwolone i ignorowane, co umożliwia grupowanie bitów dla czytelności.

Przykład dla liczby 9, której kod wewnętrzny to `0x83100000`:

```
$ ./task2_internal_to_ieee 10000011000100000000000000000000
$ ./task2_internal_to_ieee "1000 0011 0001 0000 0000 0000 0000 0000"
```

Aplikacja obsługuje dwa tryby:

1. **Argument wiersza poleceń**: 32-bitowy ciąg binarny podany bezpośrednio.
2. **Tryb interaktywny**: aplikacja wyświetla prompt i czeka na wejście.

Algorytm konwersji: kod wewnętrzny $\rightarrow$ IEEE 754

Wykorzystujemy tożsamość $\text{wartość}_{\text{wew}} = \frac{1}{2} \text{wartość}_{\text{IEEE}}$ (z sekcji 5.1). Algorytm:

1. Przelóż pola słowa wewnętrznego do układu IEEE: znak z bitu 23 na bit 31, wykładnik z bitów `[31:24]` na `[30:23]`, mantysa bez zmian.
2. Pomnóż tak uzyskaną wartość przez $\frac{1}{2}$ (zmiana nadmiaru $128 \rightarrow 127$). Wynikiem jest słowo IEEE 754.

Dla liczb znormalizowanych krok 2 to po prostu zmniejszenie pola wykładnika o 1 ($E_{b,\text{IEEE}} = E_b - 1$) przy niezmienionej mantysie. Mnożenie realizowane sprzętowo (typ `float`) jest dokładne i automatycznie odtwarza zachowanie IEEE 754 dla wszystkich pozostałych klas: znaku zera, liczb zdenormalizowanych, Inf/NaN oraz ewentualnego niedomiaru.

```

1 uint32_t internal_to_ieee754(uint32_t internal) noexcept {
2     const uint32_t S = (internal & 0x00800000u) ? 1u : 0u; // bit 23
3     const uint32_t E_b = (internal >> 24u) & 0xFFu; // bity [31:24]
4     const uint32_t mant = internal & 0x007FFFFFu; // bity [22:0]
5     // te same pola w układzie IEEE (znak 31, wykładnik 30:23, mantysa 22:0)
6     const uint32_t as_ieee = (S << 31u) | (E_b << 23u) | mant;
7     // wartosc_wew = wartosc_ieee / 2 (nadmiar 128 -> 127)
8     return float_to_bits(bits_to_float(as_ieee) * 0.5f);
9 }

```

Wyniki dla danych wzorcowych i nowych przypadków

Liczba	Wejście (kod binarny naturalny)	Kod wewn. (hex)	IEEE 754 (hex)
0	0000...0 (32 zera)	0x00000000	0x00000000
1	1000 0000 0000 ... 0000	0x80000000	0x3F800000
9	1000 0011 0001 0000 ...	0x83100000	0x41100000
65535	1000 1111 0111 1111 ...	0x8F7FFF00	0x477FFF00
65536	1001 0000 0000 ... 0000	0x90000000	0x47800000
0,5	0111 1111 0000 ... 0000	0x7F000000	0x3F000000
-1	1000 0000 1000 ... 0000	0x80800000	0xBF800000
-9	1000 0011 1001 0000 ...	0x83900000	0xC1100000

Tabela 6: Wyniki konwersji: wejście w kodzie binarnym naturalnym, wyjście IEEE 754. Ostatnie trzy wiersze demonstrują obsługę ułamków i liczb ujemnych.

Zadanie 5.3: Konwersja IEEE 754 do kodu wewnętrznego

Treść zadania

Napisać aplikację konwersji liczb zmiennoprzecinkowych zgodnych z IEEE 754 do postaci kodu "wewnętrznego".

Implementacja

Aplikacja napisana w C++17, kompilowalna na systemach Windows, macOS i Linux (clang++ lub g++). Kod źródłowy:

- `src/shared.cpp` / `src/shared.h`: funkcje wspólne (te same co w zadaniu 5.2),
- `src/task3_ieee_to_internal.cpp`: punkt wejścia (main).

Format wejścia

Użytkownik wprowadza **32-bitową wartość IEEE 754 w postaci szesnastkowej** (z przedrostkiem 0x lub bez). Aplikacja odczytuje wzorzec bitowy słowa i konwertuje go na kod wewnętrzny, wypisując wynik również w postaci szesnastkowej.

Przykłady wywołań:

```

$ ./task3_ieee_to_internal 0x41100000 # 9
$ ./task3_ieee_to_internal 0x3F000000 # 0,5
$ ./task3_ieee_to_internal 0xC1100000 # -9

```

Aplikacja obsługuje dwa tryby:

1. **Argument wiersza poleceń:** wartość szesnastkowa podana bezpośrednio.
2. **Tryb interaktywny:** aplikacja wyświetla prompt i czeka na wejście.

Zachowanie dla wszystkich klas liczb

Konwersja jest *totalna*: każda wartość IEEE 754 ma odpowiednik w kodzie wewnętrznym, a zachowanie dla każdej klasy liczb jest **takie samo jak w IEEE 754**:

Wejście IEEE 754	Wynik (kod wewnętrzny)
± 0 , liczba znormalizowana, zdenormalizowana	ta sama wartość (znak zachowany)
$\pm\infty$, NaN	$\pm\infty$, NaN
$ x \geq 2^{127}$ (powyżej zakresu znorm. wewn.)	nadmiar $\rightarrow \pm\infty$ (jak w IEEE)

Nadmiar do $\pm\infty$ przy największych liczbach IEEE 754 ($E = 127$) to dokładnie takie samo zachowanie, jakie standard przewiduje przy zawężającej konwersji (np. `double` $\rightarrow$ `float`). Nie wprowadzamy żadnej własnej reguły.

Niedomiar w tym kierunku *nie występuje*: zakres kodu wewnętrznego sięga niżej niż IEEE 754 (do 2^{-150} wobec 2^{-149}), więc każda liczba IEEE 754, łącznie ze wszystkimi zdenormalizowanymi, jest reprezentowalna. Niedomiar pojawia się dopiero w kierunku odwrotnym (zadanie 5.2), gdy najmniejsze liczby zdenormalizowane kodu wewnętrznego po podzieleniu przez 2 spadają poniżej najmniejszej liczby IEEE 754.

Algorytm konwersji: IEEE 754 $\rightarrow$ kod wewnętrzny

Symetrycznie do zadania 5.2 korzystamy z tożsamości $\text{wartość}_{\text{IEEE}} = 2 \cdot \text{wartość}_{\text{wew}}$: mnożymy wartość przez 2 (nadmiar $127 \rightarrow 128$), a następnie przekładamy pola do układu wewnętrznego (znak z bitu 31 na bit 23). Dla liczb znormalizowanych sprowadza się to do zwiększenia pola wykładnika o 1 przy niezmienionej mantysie.

```
1 uint32_t ieee754_to_internal(uint32_t ieee) noexcept {
2     // wartosc_ieee(as_ieee) = 2 * wartosc_wejscia (nadmiar 127 -> 128)
3     const uint32_t as_ieee = float_to_bits(bits_to_float(ieee) * 2.0f);
4     const uint32_t S = (as_ieee >> 31u) & 1u; // bit 31
5     const uint32_t E_b = (as_ieee >> 23u) & 0xFFu; // bity [30:23]
6     const uint32_t mant = as_ieee & 0x007FFFFFu; // bity [22:0]
7     // układ wewnętrzny: wykładnik w [31:24], znak na bit 23
8     return (E_b << 24u) | (S << 23u) | mant;
9 }
```

Mnożenie i dzielenie przez 2 realizowane sprzętowo (typ `float`) są dokładne (zmieniają jedynie pole wykładnika) i przenoszą całą semantykę IEEE 754 na kod wewnętrzny bez dodatkowego kodu obsługi przypadków szczególnych.

Wyniki dla danych wzorcowych i nowych przypadków

Liczba	IEEE 754 (wejście)	Kod wewnętrzny (wyjście)
0	0x00000000	0x00000000
1	0x3F800000	0x80000000
2	0x40000000	0x81000000
9	0x41100000	0x83100000
65535	0x477FFF00	0x8F7FFF00
65536	0x47800000	0x90000000
0,5	0x3F000000	0x7F000000
0,25	0x3E800000	0x7E000000
−1	0xBF800000	0x80800000
−9	0xC1100000	0x83900000
π	0x40490FDB	0x81490FDB

Tabela 7: Wyniki konwersji IEEE 754 $\rightarrow$ kod wewnętrzny. Ostatnie pięć wierszy demonstruje obsługę ułamków i liczb ujemnych.